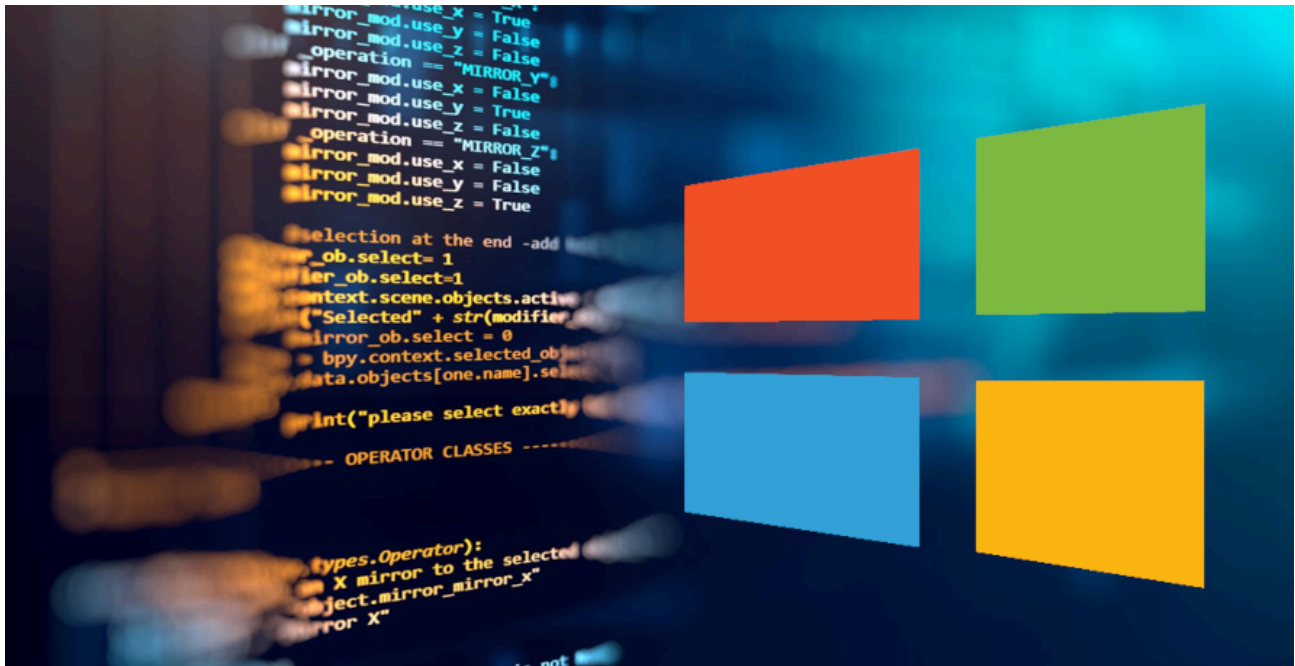


Security Descriptor Definition Language

 michaelwaterman.nl/2023/04/08/security-descriptor-definition-language

Michael Waterman

April 8, 2023



During a recent 3-month security course, I was asked on my exam to construct a Security Descriptor Definition Language (SDDL) string for a Windows object. While I was able to come up with a valid string, the experience left me curious to explore the full extent of what SDDL could do. As I delved deeper into the topic, I discovered that SDDL is a powerful tool for managing permissions on Windows objects, allowing you to fine-tune access controls with precision. In this blog post, I'll share my journey with SDDL, exploring its ins and outs, and demonstrating how you can use it to control access to various Windows objects, including services and folders.

SDDL (Security Descriptor Definition Language) is a Microsoft-specific language used to describe security descriptors for securable objects, such as files, directories, registry keys but also Active Directory objects such as organization units or even integrated DNS zones. Security descriptors contain information about access control and auditing for a securable object, including who has permission to access it and what actions they can perform.

SDDL ([nTSecurityDescriptor](#)) syntax is based on a set of text strings, each of which represents a specific security descriptor component. These components are separated by semicolons, and their order determines the overall structure of the security descriptor. The syntax includes a combination of access control entries (ACEs) and auditing entries, each of which is represented by a separate string.

The basic syntax for an SDDL string is:

```
O:S-1-5-21-xxxxxxxxxx-xxxxxxxxxx-xxxxxxxxxx-xxxxG:S-1-5-21-xxxxxxxxxx-xxxxxxxxxx-
xxxxxxxxxx-xxxxD:(A;;;FA;;;SY)(A;;;FA;;;BA)(A;;FR;;;WD)(A;;0x12019f;;;WD)
```

where:

- O stands for owner
- G stands for group
- D stands for discretionary access control list (DACL)
- S: stands for the security access control list (SACL), (not displayed)

The S-1-5-21-xxxxxxxxxx-xxxxxxxxxx-xxxxxxxxxx-xxxx strings represent security identifiers (SIDs) for the owner, group, and any user or group listed in the DACL, this part is named the header. Instead of a SID the “O:”, or “G:” can be followed by a shorthand like, “**O:DA**G:DA”. More info about these abbreviations can be found in the reference paragraph.

The (A;;FA;;;SY) string is an access control entry (ACE) that grants full access (FA) to the SYSTEM account (SY). Similarly, (A;;FA;;;BA) grants full access to the built-in Administrators group (BA), and (A;;FR;;;WD) grants read access (FR) to the Everyone group (WD).

The final ACE, (A;;0x12019f;;;WD), grants specific access rights to the Everyone group, as specified by the hexadecimal value 0x12019f. The exact access rights granted by this ACE depend on the value of the hexadecimal code.

In summary, SDDL is a language used to describe security descriptors for securable objects in Windows. It uses a set of text strings to represent security descriptor components, including access control entries and auditing entries. SIDs and ACEs are key components of SDDL and are represented by a set of flags that indicate their type and identity.

Access Control Entry

An ACE (Access Control Entry) is a component of a security descriptor that specifies a set of access rights and a security identifier (SID) that identifies a user or group to which the access rights apply. An ACE can be used to grant or deny access to a securable object.

A security descriptor can contain multiple ACEs, each of which represents a different set of access rights and applies to a different user or group. When a user attempts to access a securable object, the system checks the security descriptor to determine whether the user’s SID matches any of the SIDs listed in the ACEs. If a match is found, the access rights specified in the ACE are granted or denied, depending on whether the ACE is an allow or deny ACE.

Each ACE has the following components:

- An ACE type, which indicates whether the ACE allows or denies access.
- A set of access rights, which specifies the type of access that is granted or denied.
- A SID, which identifies the user or group to which the access rights apply.

The ACE type can be one of the following:

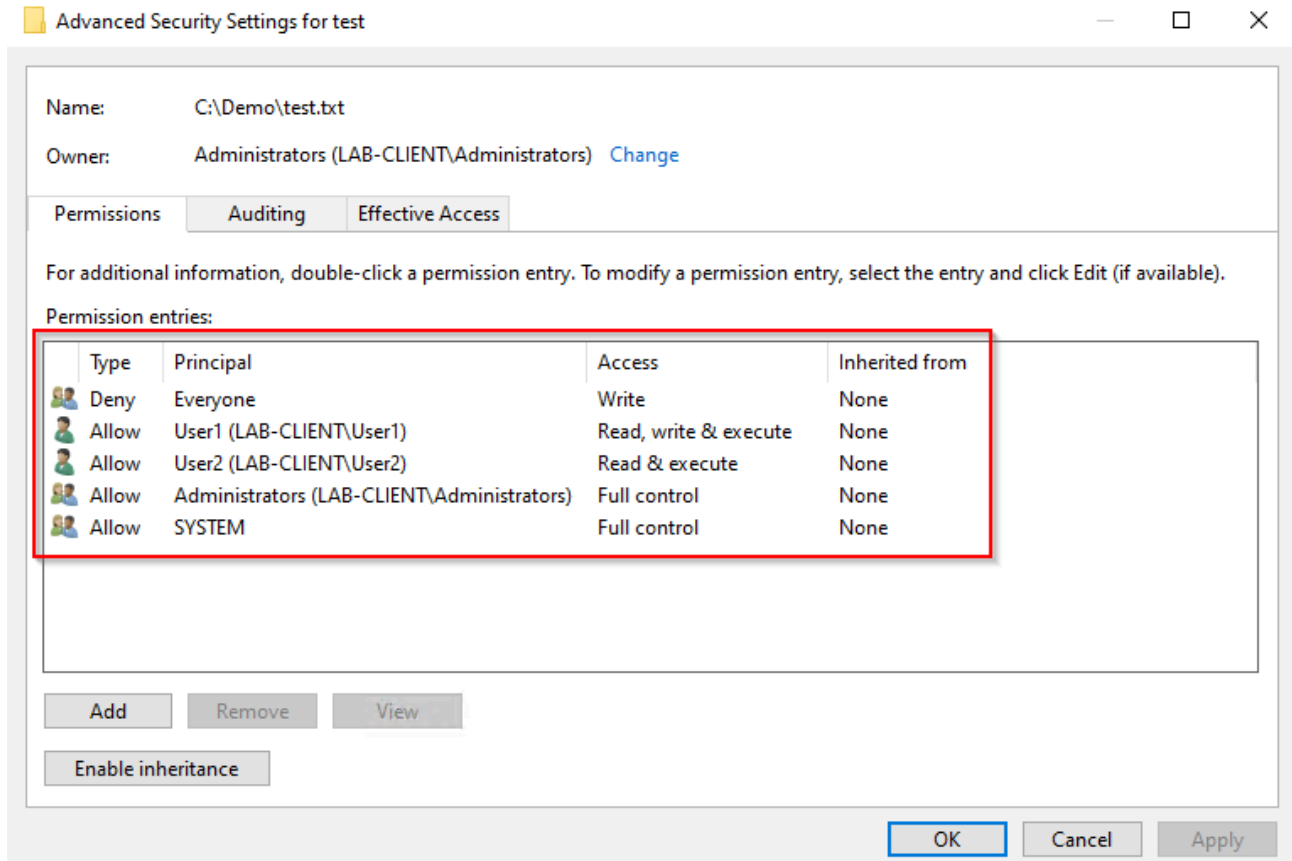
- ACCESS_ALLOWED: Allows access to the securable object.
- ACCESS_DENIED: Denies access to the securable object.
- SYSTEM_AUDIT: Generates an audit event when access is attempted.
- SYSTEM_ALARM: Generates an alarm event when access is attempted.

In addition to the ACE type, each ACE can also have flags that modify the behavior of the ACE, such as the INHERIT_ONLY flag, which specifies that the ACE should not be applied to the object itself, but only to its child objects.

ACEs are an important component of security in Windows, as they allow administrators to control access to securable objects on a granular level, specifying exactly which users or groups are allowed to perform which actions on which objects.

You can view the ACEs on a securable object using the Windows security editor, also known as the Security Descriptor Editor (SDE). Here's an example of how to view the ACEs on a file using the SDE:

1. Right-click the file for which you want to view the ACEs and select "Properties".
2. In the Properties dialog box, click the "Security" tab.
3. Click the "Advanced" button.
4. In the Advanced Security Settings dialog box, click the "Permissions" tab.
5. Here you can see the list of all the ACEs on the file, including the name of the user or group that the ACE applies to, the type of access allowed or denied, and any special conditions or flags that modify the behavior of the ACE.
6. For example, let's say you have a file called "test.txt" and you want to view the ACEs on it. If you follow the steps above, you might see something like this:
 - User1: Allow – Read & Execute, Write
 - User2: Allow – Read & Execute
 - Administrators: Allow – Full control
 - SYSTEM: Allow – Full control
 - Everyone: Deny – Write



In this example, the file “test.txt” has five ACEs, each of which applies to a different user or group. User1 has been granted Read & Execute and Write access to the file, while User2 has been granted Read & Execute access. Administrators and SYSTEM have both been granted Full control, which includes all access rights. Finally, the group Everyone has been denied Write access to the file.

Discretionary Access Control List

A DACL (Discretionary Access Control List) is a component of a security descriptor that specifies which users or groups are allowed or denied access to a securable object, such as a file, folder, or registry key. The DACL contains a list of Access Control Entries (ACEs), each of which specifies a user or group and the types of access they are allowed or denied.

The DACL is called “discretionary” because it is up to the owner of the object to decide who should have access to it. The owner can grant or revoke access to the object by modifying the DACL. The owner can also specify different levels of access for different users or groups, allowing some users to read and write the object while others can only read it.

As displayed in the paragraph on explaining what an ACE is, the picture displays all the ACEs in a list, this is the DACL.

To view or modify the DACL for a securable object, you can use the Windows security editor, also known as the Security Descriptor Editor (SDE). To access the SDE, right-click the object for which you want to view or edit the DACL, select “Properties,” and then click the “Security” tab. From there, click the “Edit” button to view or modify the DACL.

It’s important to carefully manage the DACL for sensitive objects to ensure that only authorized users can access them. By controlling access to objects, administrators can prevent unauthorized modifications, theft, or deletion of important data.

System Access Control List

A SACL (System Access Control List) is a component of a security descriptor that specifies which types of security-related events should be audited for a securable object, such as a file or folder. When a security event occurs, the system generates an audit entry that contains information about the event, such as the date and time of the event, the user or process that triggered the event, and any relevant details about the event itself.

The SACL is a list of ACEs (Access Control Entries), just like the DACL (Discretionary Access Control List) that specifies who can access the securable object. Each ACE in the SACL specifies a set of audit conditions, including the type of access being audited (such as read, write, or delete), the user or group that triggers the audit event, and any additional conditions that must be met for the audit to occur.

The SACL can be used to audit a variety of security events, such as logon attempts, file accesses, registry accesses, and network connections. By auditing these events, administrators can track user activity and identify potential security breaches or policy violations.

To view or configure the SACL for a securable object, you can use the Windows security editor, also known as the Security Descriptor Editor (SDE). To access the SDE, right-click the object for which you want to view or edit the SACL, select “Properties,” and then click the “Security” tab. From there, click the “Advanced” button and then click the “Auditing” tab to view the current SACL and make any necessary changes.

It’s worth noting that auditing can generate a significant amount of event data, so it’s important to carefully consider which events to audit and to have a plan in place for analyzing and responding to audit data.

Viewing the SDDL entries with PowerShell

Understanding the security settings of a securable object, such as a file or a registry key, is a fundamental aspect of managing access control in Windows environments. Microsoft’s Security Descriptor Definition Language (SDDL) provides a flexible and comprehensive way to express these security settings. Retrieving the SDDL of a securable object can be a

valuable tool in assessing the security posture of a system or diagnosing access issues. In this chapter, we will explore how to use PowerShell, to retrieve the SDDL of a securable object. let's get started and discover how PowerShell can help us better understand the security of our systems.

You can use the “*Get-ACL*” cmdlet in PowerShell to retrieve the security descriptor for a securable object and then extract the SDDL string from it using the “*GetSecurityDescriptorSddlForm*” method. Here's an example:

Retrieve the security descriptor for the securable object

```
# Get the Security info into a variable
$acl = Get-Acl -Path "C:\Demo\file.txt"
# Extract the SDDL string from the security descriptor
$sddl = $acl.GetSecurityDescriptorSddlForm("All")
# Display the SDDL string
$sddl
```

PowerShell

In this example, we first use *Get-Acl* to retrieve the security descriptor for the file located at *C:\path\file.txt*. We then call the *GetSecurityDescriptorSddlForm* method on the security descriptor object to extract the SDDL string. The “All” parameter specifies that we want to include all components of the security descriptor (DACL, SACL, owner, and group) in the SDDL string.

This results in the following SDDL:

```
O:S-1-5-21-210428988-3826162257-3382573153-1001G:S-1-5-21-210428988-3826162257-3382573153-513D:AI(A;ID;FA;;;BA)(A;ID;FA;;;SY)(A;ID;0x1200a9;;;BU)(A;ID;0x1301bf;;;AU)
```

let's break down this SDDL string step by step:

O: This indicates the owner of the object, in this case, it is set to “S-1-5-21-210428988-3826162257-3382573153-1001”, which is a specific user account.

G: This indicates the group that has access to the object, in this case, it is set to “S-1-5-21-210428988-3826162257-3382573153-513”, which is the “Domain Users” group.

D: This indicates the DACL (Discretionary Access Control List) of the object, which specifies which users or groups have access to the object and what kind of access they have.

Now let's break down the individual ACEs (Access Control Entries) in the DACL:

(A;ID;FA;;;BA): This ACE grants full access (FA) to the Built-in Administrators group (BA). The “ID” flag indicates that this ACE was inherited from a parent object, corresponding to

SDDL_INHERITED.

(A;ID;FA;;;SY): This ACE grants full access (FA) to the Local System account (SY). The “ID” flag indicates that this ACE was inherited from a parent object, corresponding to SDDL_INHERITED.

(A;ID;0x1200a9;;;BU): This ACE grants Read and Execute access (0x1200a9) to the Built-in Users group (BU). The “ID” flag indicates that this ACE was inherited from a parent object, corresponding to SDDL_INHERITED.

(A;ID;0x1301bf;;;AU): This ACE grants the ability to change permissions and take ownership (0x1301bf) to the Authenticated Users group (AU). The “ID” flag indicates that this ACE was inherited from a parent object, corresponding to SDDL_INHERITED.

Reference (Thanks David!) : [ACE Strings – Win32 apps | Microsoft Learn](#)

So in summary, this SDDL string sets the owner of the object to a specific user, grants full access to the Built-in Administrators and Local System accounts, grants Read and Execute access to the Built-in Users group, and grants the ability to change permissions and take ownership to the Authenticated Users group.

SDDL Strings for Access Control Entries

When working with Windows security, it's common to encounter Security Descriptor Definition Language (SDDL) strings. These strings are used to represent the security settings of various securable objects in the system, such as files, folders, and registry keys. Often, these strings include one or more Access Control Entries (ACEs) that define specific access rights and permissions for particular security principals. Understanding these ACEs is crucial for managing Windows security effectively.

In this chapter, we'll explore how to interpret SDDL strings that include ACEs. We'll break down each element of the string and explain what it represents, including the security identifier (SID) of the principal, the access mask, and the object type. By the end of this chapter, you'll have a solid understanding of how to interpret ACEs in SDDL strings, and be able to effectively manage Windows security.

Each entry in the SDDL string is separated by a semicolon ;, and each entry has a specific meaning. Here's a breakdown of what each entry means and what should be placed where:

(Type;Flags;Permission;ObjectType;InheritedObjectType;SID)

Type: This specifies the type of the ACE (Access Control Entry), and can be either “A” (allow), “D” (deny), “OA” (object allow), or “OD” (object deny).

Flags: This specifies any ACE flags, which modify the behavior of the ACE. These flags are optional, and can include the “CI” (container inherit) flag, the “OI” (object inherit) flag, the “IO” (inherit only) flag, the “ID” (inheritance from a parent object.) flag, and the “SA” (audit success) and “FA” (audit failure) flags.

Permission: This specifies the specific access rights being granted or denied by the ACE. This can be either a numeric value (in hexadecimal notation) that represents the access rights, or a shorthand name for the access rights.

ObjectType: This is an optional parameter that specifies the GUID of the object type that the ACE applies to. If this entry is not present, then the ACE applies to the default object type for the securable object.

InheritedObjectType: This is an optional parameter that specifies the GUID of the object type that the ACE is inherited from. If this entry is not present, then the ACE is not inherited.

SID: This specifies the SID (Security Identifier) of the security principal that the ACE applies to.

It’s worth noting that not all entries are required for every ACE. The Flags, ObjectType, and InheritedObjectType entries are all optional, depending on the specific use case. The Type, Permission, and SID entries are required for every ACE.

Let take our four Access Control Entries as SDDL Strings and break them down to understandable language:

- (A;ID;FA;;;BA)
- (A;ID;FA;;;SY)
- (A;ID;0x1200a9;;;BU)
- (A;ID;0x1301bf;;;AU)

(A;ID;FA;;;BA)

- **A:** This specifies that this is an Access Allowed ACE.
- **ID:** Indicates that this ACE was inherited from a parent object (corresponds to SDDL_INHERITED).
- **FA:** This specifies the permissions that are allowed, in this case, “File All Access” which includes all permissions.
- **BA:** This specifies the SID of the security principal to whom the ACE applies, in this case, the Built-in Administrators group (BA).

This ACE grants the Built-in Administrators group (BA) full access (File All Access) to the object to which this ACE applies.

(A;ID;FA;;;SY)

- **A:** This specifies that this is an Access Allowed ACE
- **ID:** Indicates that this ACE was inherited from a parent object (corresponds to SDDL_INHERITED).
- **FA:** This specifies the specific access rights that are allowed. In this case, FA stands for “File All Access”, which includes all possible access rights to the object.
- **SY:** This specifies the security principal that is allowed access. In this case, SY represents the local system account.

This ACE grants Full Control (FA) permissions to the local system account (SY) on the object it is applied to. This means that the local system account will have full permissions to modify the object and perform any actions allowed by the permissions granted. This ACE is commonly used to grant the system account permissions to perform tasks such as managing services, modifying files, or controlling system settings.

(A;ID;0x1200a9;;;BU)

- **A:** This stands for “Allow”. It specifies that the ACE grants permissions to the specified security principal (BU).
- **ID:** Indicates that this ACE was inherited from a parent object (corresponds to SDDL_INHERITED).
- **0x1200a9:** This is a hexadecimal representation of the permissions being granted
- **BU:** This specifies the security principal to which the ACE applies, in this case, the “Built-in Users” group.

This ACE grants the “Built-in Users” group (BU) the right to read, execute and list the contents of the object. One reason you might switch to using hex codes instead of abbreviations is when you need to specify a combination of access rights that doesn’t have a pre-defined abbreviation. For example, if you needed to specify the combination of read and write access but not execute access, there is no pre-defined abbreviation for this combination. In this case, you could use the hex code “0x00000002 | 0x00000020” to represent the access rights.

Another reason to use hex codes is when you’re working with programming languages that require access rights to be represented as numerical values. In this case, you would need to use the hex code representation of the access rights.

(A;ID;0x1301bf;;;AU)

- **A:** This indicates that this ACE is an “allow” entry.
- **ID:** Indicates that this ACE was inherited from a parent object (corresponds to SDDL_INHERITED).

- **0x1301bf**: This is the access mask for the ACE, represented in hexadecimal notation. The access mask indicates the specific permissions granted.
- **AU**: This indicates that the ACE applies to the “Authenticated Users” group, which includes all user accounts that have been authenticated by the system. These users could be local to the system or members of a domain.

Note! For an overview of all possible entries see the reference chapter in this blog.

Putting it into practice

Securable objects such as files, folders, and registry keys have Access Control Lists (ACLs) that define which users or groups can perform specific actions on the object. The Security Descriptor Definition Language (SDDL) is a string format used to represent the ACLs in a human-readable form. As a system administrator or security professional, it's important to be able to read, modify, and delete these SDDL entries to manage access to sensitive objects in your environment.

In this chapter, we will explore some practical examples of working with SDDL entries using PowerShell. We will start by showing you how to retrieve the SDDL for a securable object and interpret its meaning. Then, we will demonstrate how to modify SDDL entries to grant or revoke access to specific users or groups. Finally, we will show you how to delete SDDL entries to remove unnecessary access rights and improve security in your environment. By the end of this chapter, you will have a better understanding of how SDDL works and how to use it to manage access to your organization's resources.

Reading the SDDL from an object

There are multiple ways you can read the SDDL from an object, the most common, straight forward way of doing this is using PowerShell with the following syntax:

PowerShell

```
Get-Acl -Path "C:\Demo\file.txt" | Select-Object -ExpandProperty Sddl
```

PowerShell

If you need a finer grained approach you can use the following code:

PowerShell

```
$acl = Get-Acl -Path "C:\Demo\file.txt"  
$acl.GetSecurityDescriptorSddlForm("All")
```

PowerShell

The options available for “*GetSecurityDescriptorSddlForm*” are:

- None: Do not obtain any information.
- Audit: Obtain the SACL information.
- Access: Obtain the DACL information.
- Owner: Obtain the owner of the securable object.
- Group: Obtain the owner group of the securable object.
- All: Obtain all security information from the securable object.

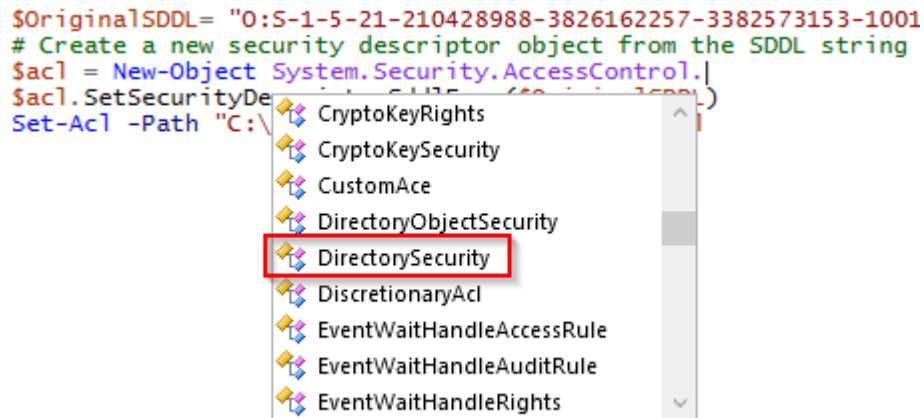
Writing to a securable object

An example of how you could manipulate a SDDL on an object with PowerShell is displayed here:

PowerShell

```
$OriginalSDDL= "O:S-1-5-21-210428988-3826162257-3382573153-1001G:S-1-5-21-210428988-3826162257-3382573153-513D:AI(A;ID;FA;;;BA)(A;ID;FA;;;SY)(A;ID;0x1200a9;;;BU)(A;ID;0x1301bf;;;AU)"
# Create a new security descriptor object from the SDDL string
$acl = New-Object System.Security.AccessControl.DirectorySecurity
$acl.SetSecurityDescriptorSddlForm($OriginalSDDL)
Set-Acl -Path "C:\Demo\file.txt" -AclObject $acl
```

PowerShell



The “*System.Security.AccessControl.DirectorySecurity*” class is created as an object and assigns it to the `$acl` variable. This object represents the access control list (ACL) for a directory, file (*FileSecurity*), or registry key (*RegistrySecurity*).

Next, the “*SetSecurityDescriptorSddlForm()*” method is called on the `$acl` object, passing in the original SDDL string as a parameter. This method sets the security descriptor of the `$acl` object based on the SDDL string.

Finally, the “*Set-ACL*” cmdlet is used to apply the modified ACL to a file located at “*C:\Demo\file.txt*”. The “*-AclObject*” parameter specifies the `$acl` object containing the modified ACL.

In summary, these four lines of PowerShell code read the SDDL of a securable object, modify the SDDL by manipulating its access control entries, and then apply the modified SDDL back to the securable object.

Removing Inheritance

Sometimes you need to remove the inherited permissions from a securable object, for example when you want to apply custom permissions to a folder or a file. In this section, we'll show you how to remove inheritance from a securable object using PowerShell. We'll also explain what inheritance is and how it works, so you can better understand how to customize permissions for your files and folders.

PowerShell

```
$File = "C:\Demo\file.txt"
$SDDLString = Get-Acl -Path $File | Select-Object -ExpandProperty Sddl
$SDDLString = $SDDLString.Replace(';ID;',',;').Replace(':AI',':PAI')
$acl = New-Object System.Security.AccessControl.DirectorySecurity
$acl.SetSecurityDescriptorSddlForm($SDDLString)
Set-Acl -Path "C:\Demo\file.txt" -AclObject $acl
```

PowerShell

The original SDDL is listed as:

```
0:BAG:S-1-5-21-792034596-1182666549-3491969430-513D:AI(A;ID;FA;;;BA)(A;ID;FA;;;SY)
(A;ID;0x1200a9;;;BU)(A;ID;0x1301bf;;;AU)
```

The modified one is set as:

```
0:BAG:S-1-5-21-792034596-1182666549-3491969430-513D:PAI(A;;;0x1301bf;;;AU)
(A;;;FA;;;SY)(A;;;FA;;;BA)(A;;;0x1200a9;;;BU)
```

The “D:PAI” part of the SDDL string you provided is called the “SDDL security information field”, which specifies the type of security information being defined in the SDDL string.

In this case, “D” specifies that the security descriptor being defined has a discretionary access control list (DACL), which controls access to the object for users and groups that are not the owner or part of the object’s primary group.

The “PAI” portion of the field specifies the inheritance and propagation flags for the security descriptor, which control how security permissions are inherited and propagated to child objects. The “P” flag indicates that permissions are inherited from parent objects, the “A” flag indicates that permissions are automatically inherited by child objects, and the “I” flag indicates that the permissions are inherited by child objects only if the objects are created within the container.

So, in summary, “D:PAI” specifies that the SDDL string is defining a discretionary access control list and specifies the inheritance and propagation flags for the security descriptor.

Pay close attention to the difference just after the owners SID, AI versus PAI.

The difference between “:PAI” and “:AI” is that “:PAI” specifies that the ACEs in the DACL are protected, while “:AI” does not. Protected ACEs cannot be modified by inheritable ACEs, while unprotected ACEs can. In other words, if an ACE is protected, it will only apply to the object it is directly assigned to, and it will not be inherited by child objects. This is useful when you want to specify explicit permissions for an object and prevent those permissions from being inherited by other objects.

So in the given SDDL string, “:PAI” indicates that the ACEs in the DACL are protected, while “:AI” would indicate that they are not. To make is simple, “AI:” means the object has inheritable ACEs, while the indication “PAI:” has explicit permissions set. Also the “ID”, indicates that the ACE is inherited from another object. An DACL can contain both ACEs with or without the inheritance flag “ID:”.

Name: C:\Demo\file.txt

Owner: Administrators (LAB-CLIENT\Administrators) [Change](#)

Permissions Auditing Effective Access

For additional information, double-click a permission entry. To modify a permission entry, select the entry and click Edit (if available).

Permission entries:

Type	Principal	Access	Inherited from
Allow	User1 (LAB-CLIENT\User1)	Read & execute	None
Allow	Administrators (LAB-CLIENT\Administrators)	Full control	C:\
Allow	SYSTEM	Full control	C:\
Allow	Users (LAB-CLIENT\Users)	Read & execute	C:\
Allow	Authenticated Users	Modify	C:\

[Add](#) [Remove](#) [View](#)

[Disable inheritance](#)

Delegating Start and Stop for a service

Managing permissions for services on Windows systems can be a challenging task, particularly when it comes to delegating the right to start or stop a service. This is where the Security Descriptor Definition Language (SDDL) can come in handy. SDDL allows for fine-grained control over permissions for various types of securable objects, including services. By understanding how SDDL works and how to modify it, it's possible to delegate start and stop rights for services to specific users or groups, while ensuring that the overall security of the system is not compromised. In this section, we'll explore some practical

examples of using SDDL to delegate start and stop rights for services, and discuss some of the potential pitfalls to watch out for.

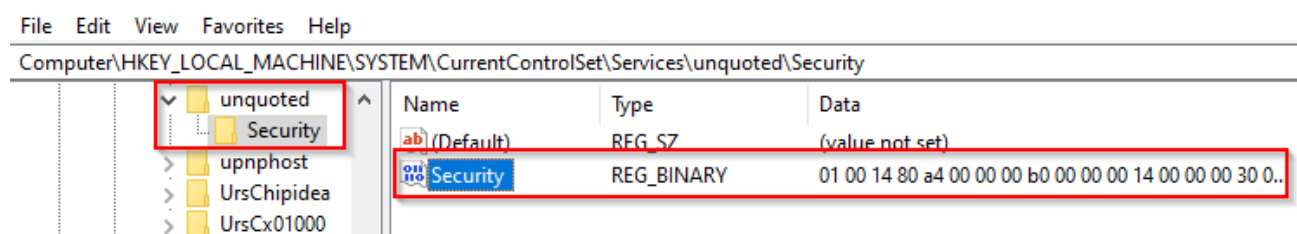
Scenario: Setting an explicit SDDL to allow a security group to start and stop a service using PowerShell. Let's use this PowerShell code.

PowerShell

```
$ServiceName = "Unquoted"
$LocalGroupName = "Service Managers"
$ServiceSDDL = sc.exe sdshow $ServiceName | where {$_.trim() -ne "" }
$NTAccountObj = New-Object System.Security.Principal.NTAccount($LocalGroupName)
$SIDLocalGroupName =
$NTAccountObj.Translate([System.Security.Principal.SecurityIdentifier]).Value
sc.exe sdset $ServiceName ($ServiceSDDL.Replace("D:(", "D:
(A;;;RPWP;;;$SIDLocalGroupName)"))
```

PowerShell

I'm sure by now that you are able to read what's going on here. There are, however, a few special things to notice. First part is this code using the "sc.exe" tool. As there are no built-in PowerShell cmd-lets (that I know of) that can manipulate the security of a service, the "sc.exe" tool is used to read and write the security descriptor. After that the security identifier of a group named "Service Managers" is retrieved and stored in the "\$SIDLocalGroupName" variable. The last line adds the group SID and the options for starting and stopping a service ("RPWP"). Where RP stands for "SERVICE_START" and WP means "SERVICE_STOP".



Note! Additional options are available in the "Reference" chapter.

Note! While Windows doesn't support managing security for services using PowerShell, an third party module can make that possible, check out [PowerShellAccessControl](#).

When a user that's part of the security group logs on they are now able to start and stop the service.

Hiding a service using SDDL

Have you ever wanted to hide a service on your Windows machine from prying eyes? Perhaps you have a custom service that runs with high privileges and you don't want unauthorized users or attackers to know about it or to be able to stop it. In this section of

the blog, we'll show you how to use SDDL to restrict access to a Windows service and effectively "hide" it from view. We'll cover the steps to modify the service's security descriptor with a new access control entry (ACE) that denies the "Query Service Configuration" permission to certain groups or users. This will prevent them from listing the service details or properties in the Services management console, the sc.exe command-line tool, or any other program that uses the Service Control Manager (SCM) API. Note that this technique is not foolproof and does not provide absolute security, but it can make it harder for attackers to find and manipulate your service, and can also serve as a defense-in-depth measure in conjunction with other security controls. Let's look at the code!

PowerShell

```
$ServiceName = "Unquoted"
$LocalGroupName = "Service Managers"
$ServiceSDDL = sc.exe sdshow $ServiceName | where {$_.trim() -ne "" }
sc.exe sdset $ServiceName ($ServiceSDDL.Replace("D:(", "D:(D;;DCLCWPDTSD;;;BA)"))
```

PowerShell

The code isn't that different from the previous one we used, except the SDDL. Here's the breakdown of the SDDL string (D;;DCLCWPDTSD;;;BA):

- *"D"* – The first part *"D"* specifies the access control entry (ACE) type as a "deny" ACE for the "object" portion of the securable object's security descriptor. The "object" portion refers to the object itself, not any child objects.
- *"DCLCWPDTSD"* – The second part *"DCLCWPDTSD"* is the access mask, which defines the specific access rights that are being denied. Each letter corresponds to a specific access right, and the letters used in this access mask stand for the following rights:
 - DC – Service Delete
 - LC – Service Query Configuration
 - WP – Service Start
 - DT – Service Interrogate
 - SD – Service Security
- *"BA"* – The third part *"BA"* specifies the security principal to which the ACE applies. In this case, *"BA"* refers to the "built-in administrators" group on the local machine. The empty fields before and after *"BA"* indicate that the ACE doesn't apply to any specific user or group in addition to the built-in administrators group. Needless to say that *"BA"* can be replaced with any SID you choose.

So in summary, the SDDL string *"(D;;DCLCWPDTSD;;;BA)"* is denying the "built-in administrators" group on the local machine access to delete, Get the configuration, start the service, obtain service information or access the security on the object portion of the securable object's security descriptor.

Conclusion

In conclusion, Security Descriptor Definition Language (SDDL) is a powerful tool used to define the security of a securable object in Windows. SDDL strings are used to specify the security descriptor of an object, which includes information about the owner, group, discretionary access control list (DACL), and system access control list (SACL).

SDDL can be used to modify or remove permissions from a securable object, and can also be used to delegate start and stop rights for services. Additionally, it can be used to remove inheritance from a securable object, which is often a challenge in enterprise environments.

Powershell provides powerful cmdlets such as “*Get-ACL*”, “*Set-ACL*”, and “*Get-SecurityDescriptorSddlForm*” that make it easy to read, modify, and delete SDDL entries for securable objects.

Understanding SDDL can be a valuable asset for IT professionals who need to manage security on Windows systems. By mastering SDDL and related PowerShell cmdlets, you can ensure that your systems are secure and that access is granted to the right users and groups.

I hope this guide has provided a helpful introduction to SDDL and its uses. Thanks for reading!

References

Available shorthand's

SDDL shorthands are used to represent security identifiers (SIDs), access control entries (ACEs), and access rights in a compact and standardized way, making it easier to understand and interpret the security attributes of system objects. This reference section aims to provide a comprehensive list of commonly used SDDL shorthand's along with their meanings and contexts of use, to help readers better understand and work with SDDL strings.

File or Folders

- FA: Full Control
- M: Modify
- RX: Read and Execute
- R: Read
- W: Write
- D: Delete
- RC: Read Control

- WDAC: Write DAC
- WO: Write Owner
- S: Synchronize
- AS: Access System Security
- GA: Generic All
- GR: Generic Read
- GW: Generic Write
- GX: Generic Execute
- RA: Read Attributes
- WA: Write Attributes
- RE: Read Extended Attributes
- WE: Write Extended Attributes
- RCX: Read, Execute, and Append Data
- DC: Delete Child
- DT: Delete Tree
- LC: List Contents
- LO: List Object
- DCX: Delete Subfolders and Files, Read and Execute

Registry Keys

- A: All Access
- AA: Allowed Ace
- BA: Built-in Administrators
- BG: Built-in Guests
- BO: Backup Operators
- BS: Batch Servers
- BU: Built-in Users
- CA: Change Permissions
- CC: Create Child
- CI: Container Inherit
- CO: Connect Network Drive
- CR: Enumerate Subkeys
- DC: Delete Child
- DCX: Delete Child Explicit
- DD: Delete
- DE: Read Control
- DG: Domain Guests
- DU: Domain Users
- EA: Execute/Traverse
- FA: File All Access
- GA: Generic All

- GE: Generic Execute
- GW: Generic Write
- GX: Generic Execute/Traverse
- H: Read
- IU: Interactive Users
- KEY_ALL_ACCESS: All Access
- KEY_CREATE_LINK: Create Link
- KEY_CREATE_SUB_KEY: Create Subkey
- KEY_ENUMERATE_SUB_KEYS: Enumerate Subkeys
- KEY_EXECUTE: Read
- KEY_NOTIFY: Notify
- KEY_QUERY_VALUE: Query Value
- KEY_READ: Read
- KEY_SET_VALUE: Write
- KEY_WOW64_32KEY: Redirected Access (32-bit)
- KEY_WOW64_64KEY: Redirected Access (64-bit)
- KEY_WRITE: Write
- LA: List Directory
- LC: Delete
- LO: Open Link
- LRC: Read Link
- LWC: Write Link
- LX: Execute File
- M: Maximum Allowed
- N: No Access
- OA: Object Access
- OC: Write Owner/Creator
- OI: Object Inherit
- OP: Read Owner/Creator
- OT: Write DAC
- OU: Write Owner
- P: Read and Execute
- PX: Read and Execute/File access
- Q: Query
- RC: Read Control
- RD: Read Permissions
- RE: Enumerate Values
- RS: Write DAC
- RU: Read Permissions
- RW: Read and Write
- RX: Read and Execute
- SD: Delete

- SP: Set Value
- SR: Enumerate Subkeys
- SW: SetValue/Create Subkey
- SYSTEM: System
- TR: Traverse Folder/Execute File
- W: Write
- WA: Write Attributes
- WD: Write Data
- WE: Write Extended Attributes
- WRC: Write Owner/Creator
- WS: Write Owner
- X: Execute

Windows Services

- GA: Generic All
- GW: Generic Write
- GX: Generic Execute
- RC: Read Control
- RD: Read Permissions
- WO: Write Owner
- WD: Write DAC
- S: Synchronize
- CC: Service Change Configuration
- DC: Service Delete
- LC: Service Query Configuration
- LO: Service List Object
- SD: Service Security
- CR: Service Start
- FA: Service Stop
- SP: Service Pause/Continue
- DT: Service Interrogate
- RP: Service Net Bind (Deprecated in Windows Server 2008 and Windows Vista)
- WP: Service Start
- D: Delete access
- RC: Read control
- WDAC: Write DAC
- WO: Write owner
- GA: Generic all access
- RP: Service Net Bind
- SD: Service Security
- SS: Service Stop

- LO: Service list object

Identities for ACE object types

- AU: Authenticated Users
- BA: Built-in Administrators
- BG: Built-in Guests
- BO: Backup Operators
- BU: Built-in Users
- CA: Certificate Server Administrators
- CG: Creator Group
- CO: Creator Owner
- DA: Domain Administrators
- DC: Domain Computers
- DD: Domain Controllers
- DG: Domain Guests
- DU: Domain Users
- EA: Enterprise Administrators
- ED: Enterprise Domain Controllers
- IU: Interactive Users
- LA: Local Administrator
- LG: Local Guest
- LS: Local Service
- LU: Network Logon User
- MO: Creator Owner Server
- MU: Creator Group Server
- NO: Network Configuration Operators
- NS: Network Service
- NU: Network
- OW: Owner Rights
- PS: Personal Self
- PU: Power Users
- RC: Restricted Code
- RE: Restricted Network
- RO: Replica Server Operators
- RU: Alias to allow previous Windows 2000
- SA: Schema Administrators
- SI: System
- SO: Server Operators
- SU: Service Logon User
- S-#: Security Identifier (SID)
- WD: Everyone

- WG: Windows Authorization Access Group
- WO: Well-known group object
- WR: World Access Group
- YS: Terminal Server Users
- VA: Virtual Account
- UI: NTLM Authentication

there are more securable objects in Windows that have their own lists of SDDL shorthand's. Some examples include:

- Named Pipes
- Network shares
- Printer queues
- Process objects
- WMI namespaces

The list of available shorthands and their meanings may differ depending on the securable object. It's always best to consult the Microsoft documentation or seek expert advice for the specific object you're working with.

Official Documentation

[Security Descriptor Definition Language – Win32 apps | Microsoft Learn](#)

[Security Descriptor String Format – Win32 apps | Microsoft Learn](#)

[ACCESS_MASK \(Winnt.h\) – Win32 apps | Microsoft Learn](#)

[Access Mask Format – Win32 apps | Microsoft Learn](#)

[ACE Strings – Win32 apps | Microsoft Learn](#)